ЗАНЯТИЕ 4

Планирование траекторий и кинематическое управление

Законы движения и сплайны, траектория схвата, оптимальная параметризация, управление по скорости и особые точки

Зенкевич С. Л., Ющенко А. С. Основы управления манипуляционными роботами. — МГТУ им. Н. Э. Баумана, 2004. Глава 4

90 минут · две части по 45

Курс «Управление роботами» · Кинематика манипуляционных роботов

План занятия

1. Кейс «горелка по фланцу»: где в системе управления работает кинематическое управление?
2. Как перевести сочленения из точки в точку без удара и синхронно?
3. Чем многочлены и S-кривая лучше трапеции и когда это действительно важно?
4. Как провести сочленение через много точек: кубический сплайн и растяжение времени
5. Как вести схват по линии: обратная задача в узлах, цепочка целей, ошибка хорды
6. Как поворачивать схват и как пройти путь за наименьшее время?
7. Линеаризованный алгоритм: одна итерация Ньютона на каждый такт управления
8. Управление по скорости с обратной связью по положению: устойчивость и выбор коэффициента
9. Что делать рядом с особой точкой: демпфирование, замедление, застревание
10. Управление по ускорению и по силе: мосты к динамике
11. От формул к контроллеру робота: сводка методов и контрольный список кейса

Синие — первая часть · янтарные — вторая часть

Что нужно помнить с прошлых занятий

Сегодня мы опираемся на шесть результатов трёх прошлых занятий; освежим их, прежде чем двигаться дальше.

Понятие (занятие)	Что это простыми словами	Где пригодится сегодня
Однородная матрица T (занятие 1)	Поза схвата одной матрицей 4×4: поворот R и положение p	Траектория схвата T(t) между двумя заданными позами
Ось k и угол α, формула Родрига (занятие 1)	Любой поворот — это поворот на угол α вокруг единичной оси k	Плавный поворот схвата по кратчайшему пути
Обратная задача о положении (занятие 2)	По позе схвата находят углы; решений несколько, это разные ветви	Углы в узлах окружности без перескока на другую ветвь
Метод Ньютона $q \leftarrow q + J^{+}e$ (занятие 2)	Линеаризуют около q и шагают; у цели ошибка за шаг возводится в квадрат	Цепочка близких целей: невязка $6{,}5\cdot10^{-2} \rightarrow 2{,}5\cdot10^{-3} \rightarrow 4\cdot10^{-6}$ м
Матрица Якоби $\mathcal{V} = J\dot{q}$, J^{+}, демпфирование (занятие 3)	J связывает скорости сочленений и схвата; у особой точки теряет ранг	Управление по вектору скорости и проход мимо особой точки
Статика $\tau = J^{T}F$ (занятие 3)	Моменты в сочленениях для силы F на схвате даёт транспонированная J	Управление по силе и алгоритм с транспонированной матрицей Якоби

Сокращения и обозначения занятия

Все сокращения со слайдов собраны здесь; к таблице удобно возвращаться при чтении PDF.

Обозначение	Расшифровка	Смысл в одной фразе
PTP, LIN, CIRC	Point-to-Point, Linear, Circular — команды движения робота	«Из точки в точку», «по прямой», «по дуге» в языках роботов
TOPP, TOPP-RA	Time-Optimal Path Parameterization; RA — Reachability Analysis	Оптимальная по времени параметризация пути и её быстрый алгоритм
TOTG, Ruckig	Time-Optimal Trajectory Generation; Ruckig — имя библиотеки	Параметризация по времени в MoveIt 2 и сглаживание с пределом рывка
CLIK, RMRC	Closed-Loop Inverse Kinematics; Resolved Motion Rate Control	Управление по скорости с обратной связью по положению и без неё
SLERP, SO(3), SE(3)	Spherical Linear Interpolation; группы поворотов и движений	Поворот по кратчайшему пути между двумя ориентациями
QP	Quadratic Programming — квадратичное программирование	Выбор скоростей сочленений как малая задача оптимизации в такте
ROS 2, OMPL; KUKA, ABB, FANUC, UR	Robot Operating System 2; Open Motion Planning Library; производители	Программная среда, библиотека поиска пути и марки роботов

Сокращения и обозначения занятия (продолжение)

Буквы со слайдов; везде они значат одно и то же, как в таблице обозначений курса.

Обозначение	Расшифровка	Смысл в одной фразе
$\lambda(t)$, $d\lambda/dt$; s, $\dot{s}$	Параметр пути от 0 до 1 и профиль скорости; s — тот же параметр в TOPP	Путь задают отдельно, закон движения вдоль него — отдельно
T, $\bar{t} = t/T$, t_a, t_j; $\dot{q}^{max}$, $\ddot{q}^{max}$, κ	Длительность, t/T, разгон, рост ускорения; пределы привода; растяжение	Растяжение в κ раз делит скорости на κ, ускорения — на κ^2
h_k, g_k; ε_{ch}, ρ, N_k	Шаг и правая часть сплайна; ошибка хорды, радиус, число узлов	$\varepsilon_{ch} = \rho(1 - \cos(\Delta\varphi/2))$ задаёт густоту узлов на окружности
R_o, R_f; k, α; log, exp	Начальная и конечная ориентации; ось и угол; логарифм и экспонента	$R(t) = R_o \exp(\lambda \log(R_o^T R_f))$ — поворот по кратчайшему пути
$\mathcal{V} = (\omega; v)$, $\mathcal{V}^*$, $e = (\delta\varphi; \Delta p)$, K, Δt, q_m	Скорость схвата и её команда, ошибка позы, коэффициент, такт, датчики	Звёздочка — заданное значение, индекс m — измеренное
μ, σ_i, β; $F = (m; f)$, τ	Демпфирование, сингулярные числа J, масштаб; силовой винт и моменты	$\tau = J^T F$: силу на схвате дают моменты в сочленениях
δq, δs	Малые (виртуальные) перемещения сочленений и схвата	На них считают работу в принципе виртуальных перемещений: $\tau = J^T F$

ЧАСТЬ 1

Траектории в пространстве сочленений

Сначала и задание, и исполнение живут в пространстве сочленений: строим гладкие законы движения из точки в точку и через много точек.

1 Сквозной кейс: горелка по фланцу

Чтобы разговор был предметным, возьмём производственную задачу; каждый раздел лекции закроет одно из её требований.

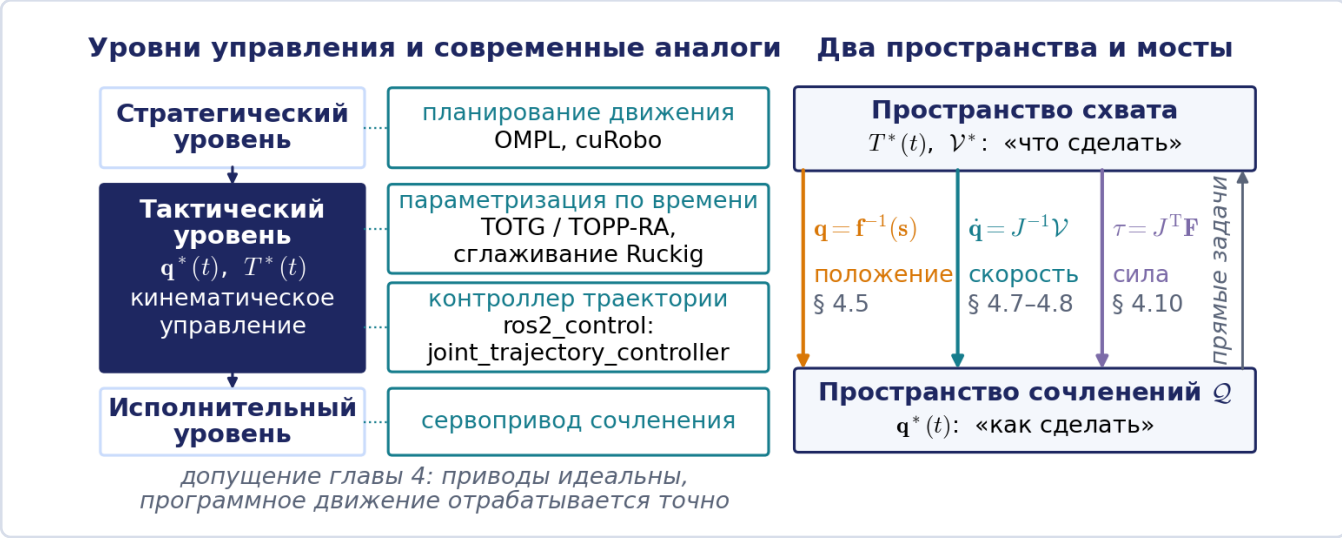
- Робот ведёт сварочную горелку по окружности фланца с заданной скоростью, и к движению предъявляют четыре требования.
- Горелка трогается и останавливается без удара, иначе механика раскачается: это законы движения и S-кривая, разделы 4.2–4.4.
- Окружность остаётся окружностью, а не многоугольником: узлы, обратная задача и ошибка хорды, разделы 4.5–4.6.
- Схват держит траекторию при неидеальных приводах: обратная связь по положению схвата, разделы 4.7–4.8.
- Рядом с особой конфигурацией робот не застревает и не требует невозможных скоростей: раздел 4.9.

Кейс

Упрощённый вариант этой задачи — второе домашнее задание: схват робота обходит окружность. В конце занятия мы соберём требования в контрольный список и проверим, каким разделом закрыто каждое.

1 Кинематическое управление: уровни и современный конвейер движения

Кинематическое управление — тактический уровень системы; у каждого уровня сегодня есть узнаваемый программный аналог.



На схеме: OMPL — Open Motion Planning Library; TOTG — Time-Optimal Trajectory Generation; TOPP-RA — оптимальная параметризация пути.

- **Кинематическое управление** считает приводы идеальными, поэтому о гладкости команд приходится заботиться уже на этом уровне.
- Сначала планировщик ищет бесстолкновительный путь, затем путь параметризуют по времени с учётом пределов приводов — разделы 4.2–4.6.
- Контроллер траектории в ros2_control соединяет узлы многочленами (раздел 4.3), а сервопривод отрабатывает их в каждом сочленении.
- Задание дано в координатах схвата, а приводы понимают углы; три моста справа дают управление по положению, по скорости и по силе.

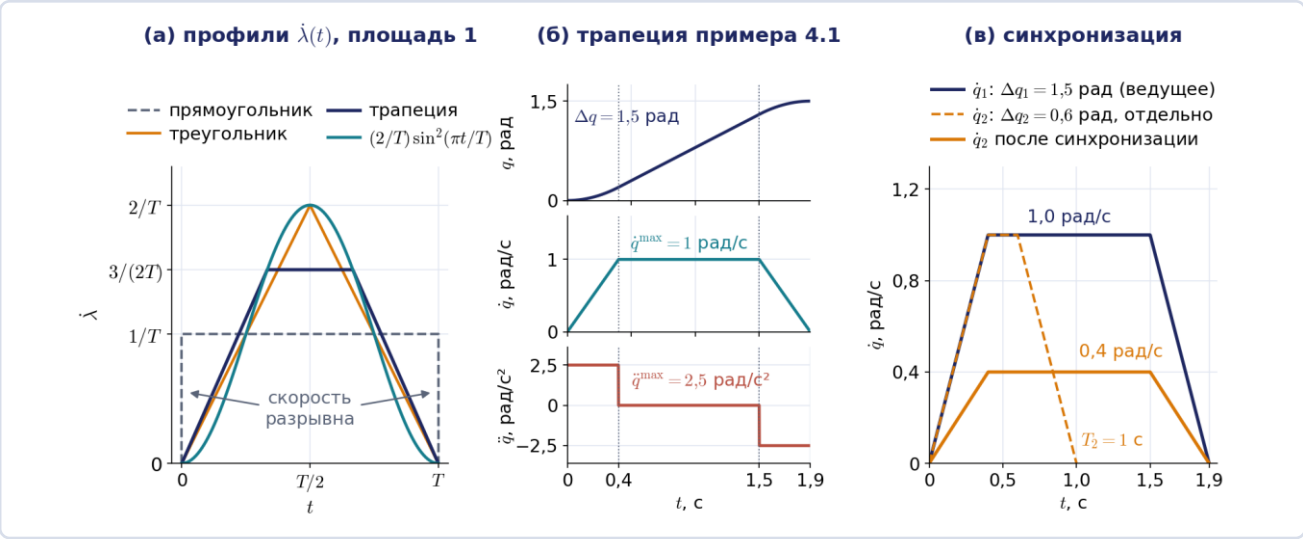
$$\mathbf{q} = \mathbf{f}^{-1}(\mathbf{s})$$

$$\dot{\mathbf{q}} = J^{-1}(\mathbf{q})\,\boldsymbol{\mathcal{V}}$$

$$\boldsymbol{\tau} = J^{\mathrm{T}}(\mathbf{q})\,\mathbf{F}$$

2 Из точки в точку: путь отдельно, закон движения отдельно

Движение с постоянной скоростью требует бесконечного ускорения при старте; спасает раздельное задание пути и закона движения.



Панели: (а) четыре профиля единичной площади, (б) трапеция, (в) синхронизация двух сочленений. В учебнике — формулы (4.4)–(4.8).

- Наивный закон $q(t) = q^0 + \dot q \cdot (t - t_0)$ даёт скачок скорости при старте и остановке, то есть бесконечное ускорение и удар по механике.
- Путь оставляют прямым, а параметр $\lambda(t)$ растёт от 0 до 1; его производная $d\lambda/dt$ — **профиль скорости**, площадь под ним равна 1.
- У прямоугольного профиля скорость разрывна, у треугольника и трапеции в изломах скачет ускорение; синусный профиль гладок, но пик выше.
- Профиль общий для всех сочленений: они трогаются и останавливаются вместе, а точка в пространстве сочленений Q идёт по прямой.

$$\mathbf{q}(t) = \mathbf{q}^0 + \lambda(t)\,(\mathbf{q}^1 - \mathbf{q}^0)$$

$$\lambda(t_0) = 0, \quad \lambda(t_1) = 1, \quad \int_{t_0}^{t_1} \dot\lambda\, dt = 1$$

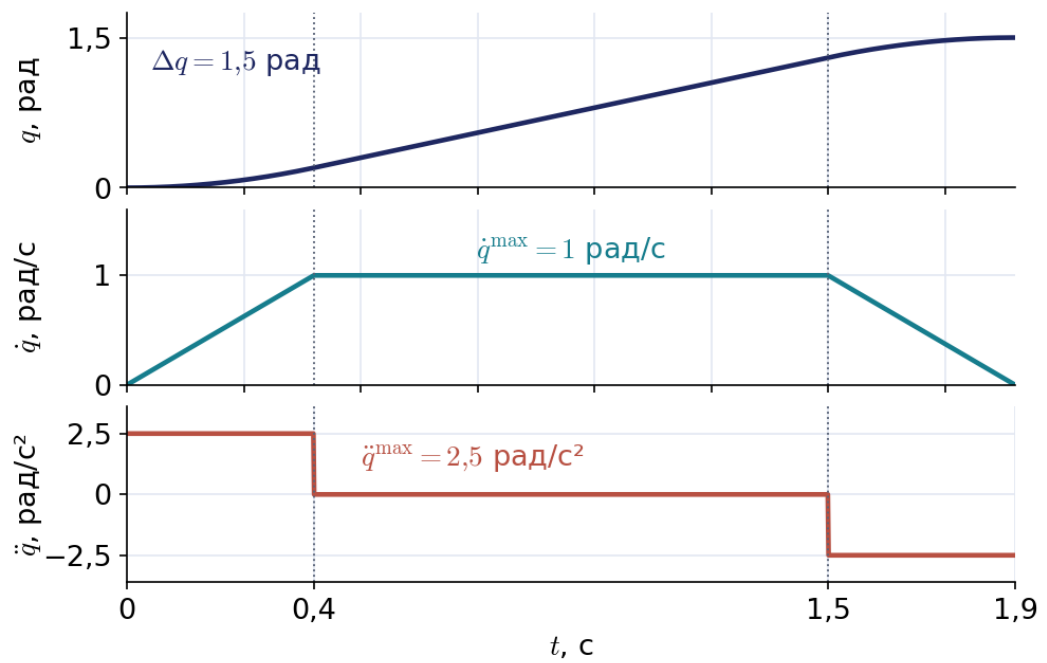
2 Трапеция: времена по пределам привода

В учебнике параметры трапеции назначены произвольно; на практике их считают по предельной скорости и ускорению привода.

- Перемещение Δq сочленение проходит так: разгон с предельным ускорением $\ddot{q}^{max}$ за время t_a, полка с предельной скоростью $\dot{q}^{max}$, торможение.
- Пример 4.1: Δq = 1,5 рад, $\dot{q}^{max}$ = 1 рад/с, $\ddot{q}^{max}$ = 2,5 рад/с²; тогда t_a = 0,4 с и T = 1,9 с.
- При Δq = 0,3 рад полка не возникает: профиль треугольный, t_a ≈ 0,35 с, T ≈ 0,69 с, пик скорости 0,87 рад/с.
- Трапеция оптимальна по быстродействию: она всё время идёт на одном из пределов, поэтому быстрее при тех же пределах не получится.

$$t_a = \frac{\dot{q}^{\max}}{\ddot{q}^{\max}}, \qquad T = \frac{\Delta q}{\dot{q}^{\max}} + t_a$$

$$\Delta q < \frac{(\dot{q}^{\max})^2}{\ddot{q}^{\max}}: \quad t_a = \sqrt{\frac{\Delta q}{\ddot{q}^{\max}}}, \quad T = 2t_a$$



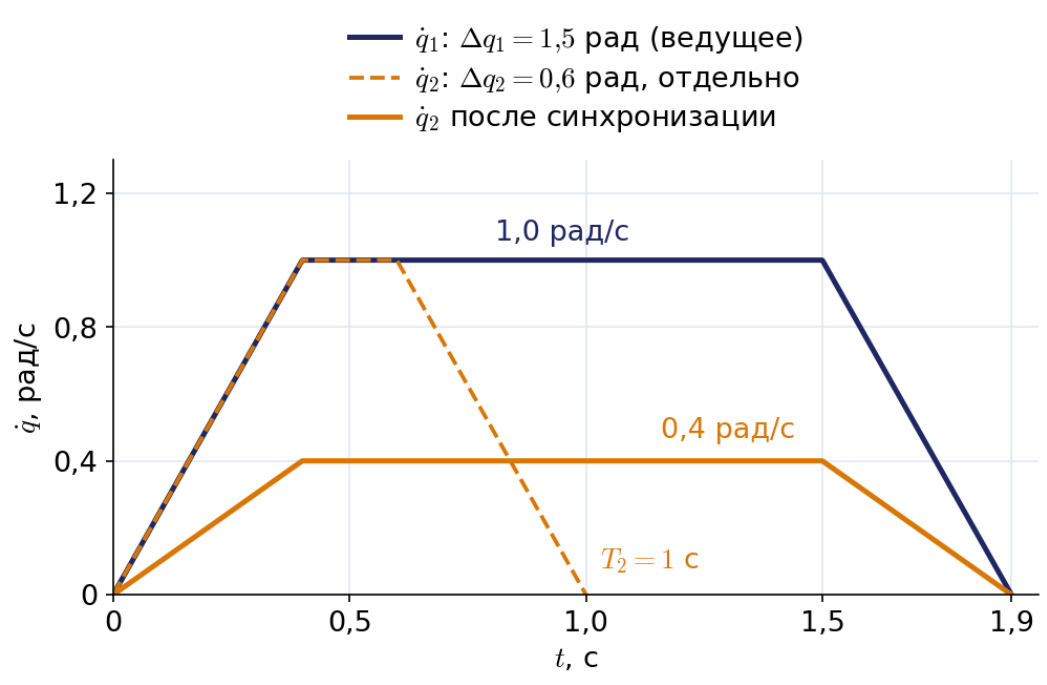
2 Синхронизация сочленений по ведущему

Если считать каждое сочленение отдельно, они закончат в разное время, и точка в пространстве $\mathcal{Q}$ пойдёт по ломаной.

- Общее время разгона берут наибольшим из собственных времён $t_{a,i}$, а общее время T — по самому медленному, ведущему сочленению.
- Затем скорости полок и ускорения пересчитывают под общие времена; ни один предел при этом не нарушается.
- Пример: второе сочленение с $\Delta q_2 = 0{,}6$ рад само закончило бы за 1 с, а после синхронизации идёт 1,9 с со скоростью 0,4 рад/с.
- Если даже ведущему не хватает времени на полку, берут $T = 2 \cdot t_a$: профиль треугольный, пределы соблюдены, но время не обязательно минимально.

$$t_a = \max_i t_{a,i}, \qquad T = \max\left(\max_i \frac{\Delta q_i}{\dot{q}_i^{\max}} + t_a,\ 2t_a\right)$$

$$\dot{q}_i = \frac{\Delta q_i}{T - t_a}, \qquad \ddot{q}_i = \frac{\dot{q}_i}{t_a}$$



? Найдите ошибку

Два сочленения: $\Delta q = (0{,}1;\ 1{,}5)$ рад, пределы скорости (1; 1) рад/с, пределы ускорения (0,5; 100) рад/с². В треугольном случае программа берёт время по одним ускорениям: $T = 2 \cdot \max \sqrt{(\Delta q / \ddot{q}^{max})} \approx 0{,}89$ с. Что произойдёт?

A Всё в порядке: треугольный профиль всегда допустим

B Первое сочленение превысит свой предел ускорения 0,5 рад/с²

C Второму сочленению придётся развить 3,35 рад/с при пределе 1 рад/с

D Сочленения закончат движение в разное время

? Найдите ошибку

Два сочленения: $\Delta q = (0{,}1;\ 1{,}5)$ рад, пределы скорости (1; 1) рад/с, пределы ускорения (0,5; 100) рад/с². В треугольном случае программа берёт время по одним ускорениям: $T = 2 \cdot \max \sqrt{(\Delta q / \ddot{q}^{max})} \approx 0{,}89$ с. Что произойдёт?

A Всё в порядке: треугольный профиль всегда допустим

B Первое сочленение превысит свой предел ускорения 0,5 рад/с²

C Второму сочленению придётся развить 3,35 рад/с при пределе 1 рад/с

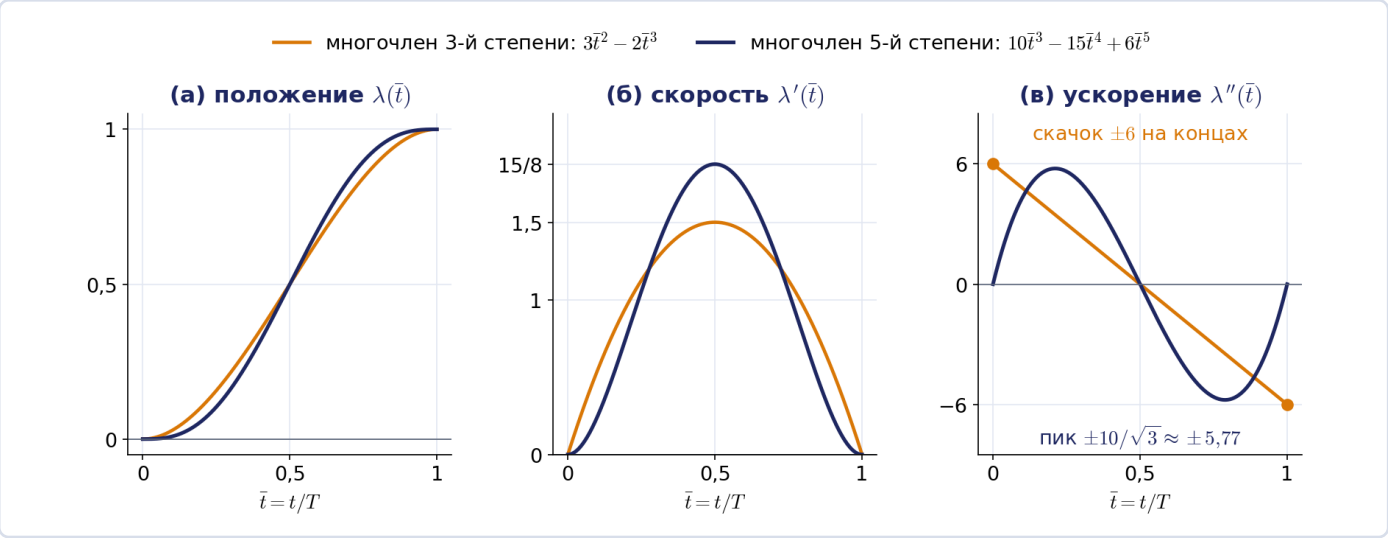
D Сочленения закончат движение в разное время

ОТВЕТ

Вариант C. Формула забыла пределы скорости: второму сочленению самому нужно не меньше 1,51 с. Верное правило даёт $t_a = 2$ с, $T = 2 \cdot t_a = 4$ с и полки 0,05 и 0,75 рад/с.

3 Многочлены третьей и пятой степени

Второй способ сгладить старт — искать закон движения многочленом и подчинить его коэффициенты условиям на концах.



Многочлен пятой степени минимизирует интеграл квадрата рывка, кубический — интеграл квадрата ускорения (вариационный смысл, конспект § 4.3).

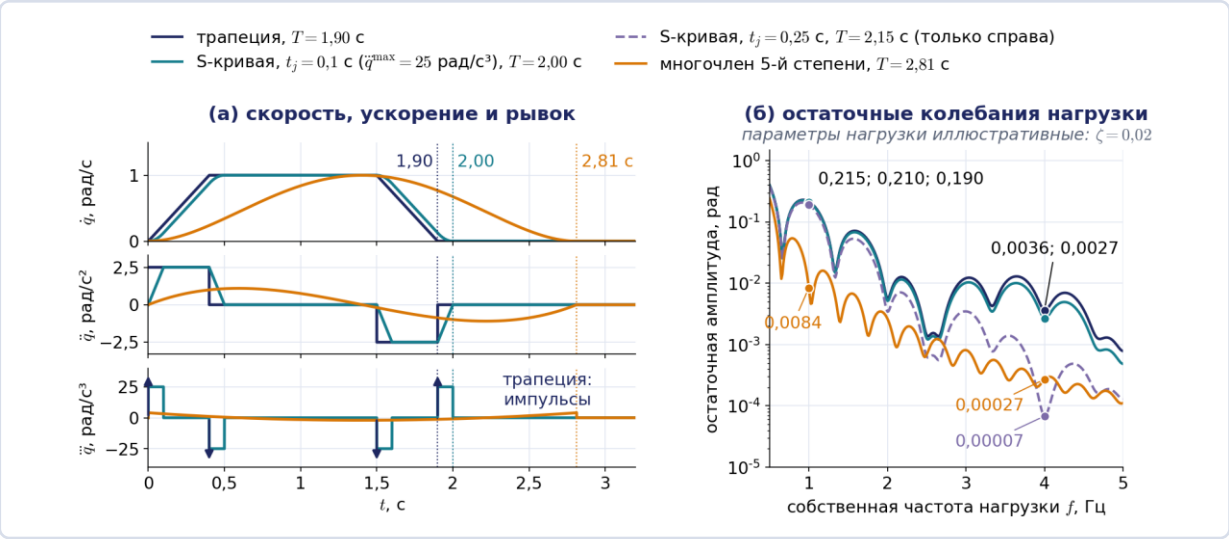
- Шесть условий (положение, скорость и ускорение на двух концах) дают многочлен пятой степени; Δ = $q^1 - q^0$, t̄ — безразмерное время.
- Пик скорости равен 15/8·Δ/T, пик ускорения 5,77·Δ/T²; у кубического многочлена ускорение на концах скачет на ±6·Δ/T², как у трапеции.
- Пример 4.2: при пределах примера 4.1 нужно T = 2,81 с против 1,9 с у трапеции — почти в полтора раза дольше.
- Контроллер траектории ros2_control строит между узлами такие же многочлены: степени 1, 3 или 5 в зависимости от заданных величин.

$$q(t) = q^0 + \Delta\left(10\bar{t}^3 - 15\bar{t}^4 + 6\bar{t}^5\right), \quad \bar{t} = \frac{t}{T}$$

$$T \geq \max\left(\frac{15}{8}\frac{\Delta}{\dot{q}^{\max}},\ \sqrt{5{,}77\frac{\Delta}{\ddot{q}^{\max}}}\right)$$

3 Ограничение рывка: S-кривая

Многочлен пятой степени платит почти секундой; промышленность добивается гладкости иначе — ограничивает третью производную, рывок.



Справа: остаточная амплитуда упругой нагрузки с собственной частотой f после остановки; параметры нагрузки иллюстративные.

- **Рывок** — третья производная положения; его скачок мгновенно меняет силы в редукторе, раскачивает упругие моды и растит ошибку слежения.
- S-кривая из семи участков: ускорение нарастает с предельным рывком, держится на пределе и спадает; затем полка и зеркальное торможение.
- Пример 4.1: трапеция 1,90 с, S-кривая 1,95 / 2,00 / 2,15 с при рывке 50 / 25 / 10 рад/с³, многочлены 5-й и 7-й степени 2,81 и 3,28 с.
- При $t_j \le t_a$ S-кривая — трапеция с ускорением, усреднённым окном t_j: остаток колебаний умножается примерно на $|\sin(\pi \cdot f \cdot t_j)/(\pi \cdot f \cdot t_j)|$.

$$t_j = \frac{\ddot{q}^{\max}}{\dddot{q}^{\max}}, \qquad T = \frac{\Delta q}{\dot{q}^{\max}} + \frac{\dot{q}^{\max}}{\ddot{q}^{\max}} + t_j$$

3 Что даёт гладкость: остаточные колебания нагрузки

Посадим на звено упругую нагрузку и сравним, насколько каждый закон движения раскачивает её после остановки.

Закон движения (данные примера 4.1)	Время T, с	Остаток на моде 1 Гц, рад	Остаток на моде 4 Гц, рад
Трапеция	1,90	0,215	0,0036
S-кривая, t_j = 0,1 с	2,00	0,210	0,0027
S-кривая, t_j = 0,25 с	2,15	0,190	0,00007
S-кривая с пределами 1 рад/с², 2,5 рад/с³ (t_j = 0,4 с; окно 1 с)	2,90	0,004	0,00006
Многочлен пятой степени	2,81	0,0084	0,00027
Трапеция с двухимпульсным формирователем на 1 Гц	2,40	≈ 0	0,0032

Модель нагрузки: $\theta'' + 2\zeta\cdot\omega_n\cdot\theta' + \omega_n^2\cdot\theta = -\ddot{q}(t)$, $\zeta = 0{,}02$, параметры иллюстративные. Формирователь (input shaping, Сингер и Сиринг, 1990) складывает команду трапеции с копией, задержанной на 0,50 с, с весами 0,516 и 0,484.

? Кейс: длинная горелка на фланце

На фланце длинная горелка с первой собственной частотой около 4 Гц. После остановки её нельзя раскачивать, а время терять не хочется. Какой закон выбрать для перемещения из примера 4.1?

A Многочлен пятой степени, T = 2,81 с

B S-кривую с t_j = 0,25 с, T = 2,15 с

C Трапецию с формирователем, настроенным на 1 Гц, T = 2,40 с

D Обычную трапецию, T = 1,90 с: на 4 Гц остаток и так мал

? Кейс: длинная горелка на фланце

На фланце длинная горелка с первой собственной частотой около 4 Гц. После остановки её нельзя раскачивать, а время терять не хочется. Какой закон выбрать для перемещения из примера 4.1?

A Многочлен пятой степени, T = 2,81 с

B S-кривую с t_j = 0,25 с, T = 2,15 с

C Трапецию с формирователем, настроенным на 1 Гц, T = 2,40 с

D Обычную трапецию, T = 1,90 с: на 4 Гц остаток и так мал

ОТВЕТ

Вариант B: остаток 0,00007 рад — в 50 раз меньше, чем у трапеции, ценой 0,25 с. Многочлен даёт 0,00027 рад и теряет 0,9 с, а формирователь на 1 Гц моду 4 Гц почти не гасит.

4 Обход многих точек: сплайн вместо одного многочлена

Через сотни точек контура не проводят один многочлен высокой степени: между узлами он раскачивается.



- Многочлен 12-й степени через 13 узлов уходит у краёв от плавной кривой больше чем на 3 рад — это **явление Рунге** из численного анализа.
- **Кубический сплайн** на каждом отрезке — многочлен не выше третьей степени, а q, q′ и q″ непрерывны во всех узлах.
- На n отрезках 4n коэффициентов и 4n − 2 условия, поэтому два условия свободны: их задают на концах (скорости или ускорения).
- Теорема Холладея: среди всех дважды гладких функций через те же узлы естественный сплайн минимизирует интеграл $(q'')^2$, отсюда его плавность.

4 Система для вторых производных сплайна

Вторая производная кубического многочлена линейна, поэтому всё неизвестное в сплайне — это значения q'' в узлах.

$$\alpha_k\, q''_{k+1} + 2\, q''_k + (1-\alpha_k)\, q''_{k-1} = g_k$$

$$\alpha_k = \frac{h_k}{h_k + h_{k-1}}, \qquad g_k = \frac{6}{h_k + h_{k-1}} \left(\frac{q_{k+1} - q_k}{h_k} - \frac{q_k - q_{k-1}}{h_{k-1}} \right)$$

- Здесь q''_k — вторая производная в узле k, h_k — шаг между узлами, α_k — вес, g_k — утроенный разностный аналог второй производной данных.
- Смысл (после деления на 3): взвешенное среднее q'' в трёх соседних узлах, веса в сумме 1, равно второй производной по данным.
- В каждой строке диагональ 2 больше суммы остальных (не больше 1), а такая матрица невырождена: решение есть и единственно.
- Трёхдиагональную систему решают прогонкой за число операций, пропорциональное n; на практике — scipy CubicSpline с краевыми условиями.

Вывод — приложение 4.A конспекта; в учебнике — формулы (4.21)–(4.24), правая часть там обозначена M_k.

4 Метод прогонки и пример 4.4

Прогонка — метод Гаусса для трёхдиагональной матрицы; разберём её на сплайне через четыре узла.

- Решение ищут в виде $q''_k = c_k \cdot q''_{k+1} + d_k$: прямой ход слева направо считает c_k и d_k, обратный ход справа налево — сами q''_k.
- Благодаря диагональному преобладанию знаменатели не обращаются в нуль, а $|c_k| \le 1$, поэтому ошибки округления не нарастают.
- Пример 4.4: узлы t = 0, 1, 2, 3 с, q = 0; 1; 1,5; 3 рад; естественный сплайн даёт q'' = (0; −1,2; 1,8; 0), но стартует со скоростью 1,2 рад/с.
- С нулевыми скоростями на концах q'' = (4,4; −2,8; 3,8; −6,4) рад/с², пик скорости около 2,0 рад/с при t ≈ 2,37 с; так же считает CubicSpline.

4 Сплайн на практике: пределы, растяжение времени, замкнутый контур

Построенный сплайн ещё надо проверить по пределам приводов; если они нарушены, проще всего растянуть время.

- Ускорение сплайна на отрезке линейно, поэтому его экстремумы в узлах; скорость проверяют ещё во внутренней точке, где q'' = 0.
- Растяжение интервалов в κ раз делит скорости на κ, ускорения на κ²: сплайн через 13 узлов с пиками 1,23 рад/с и 4,19 рад/с² требует κ = 1,29.
- Сплайн строится заранее (off-line): прогонке нужны все узлы; онлайн-генераторы вроде Ruckig пересчитывают траекторию в каждом такте.
- Для многократного обхода замкнутого контура берут периодический сплайн: скорость и ускорение в начале и в конце совпадают.

$$\kappa = \max\left(1,\ \max_i \frac{\max|\dot{q}_i|}{\dot{q}_i^{\max}},\ \max_i \sqrt{\frac{\max|\ddot{q}_i|}{\ddot{q}_i^{\max}}}\right)$$

Первая часть: что нужно помнить

Пять мыслей первой части, на которые опирается вторая.

- Путь и закон движения задают раздельно: $q = q^{o} + \lambda(t)(q^{1} - q^{o})$, а гладкость движения определяет только профиль скорости $d\lambda/dt$.
- Трапецию считают по пределам привода, $t_a = \dot{q}^{max}/\ddot{q}^{max}$ и $T = \Delta q/\dot{q}^{max} + t_a$; несколько сочленений подстраивают под ведущее.
- Гладкость нужна против конкретных собственных частот: S-кривая удлиняет движение лишь на t_j, а многочлен пятой степени — почти в полтора раза.
- Через много точек проводят кубический сплайн, а не один многочлен высокой степени; его находят из трёхдиагональной системы за O(n) операций.
- Если сплайн нарушает пределы, время растягивают в κ раз: скорости делятся на κ, ускорения — на κ^2.

П Е Р Е Р Ы В

Конец первой части

Мы научились строить гладкие законы движения в пространстве сочленений. После перерыва задание будет дано в координатах схвата.

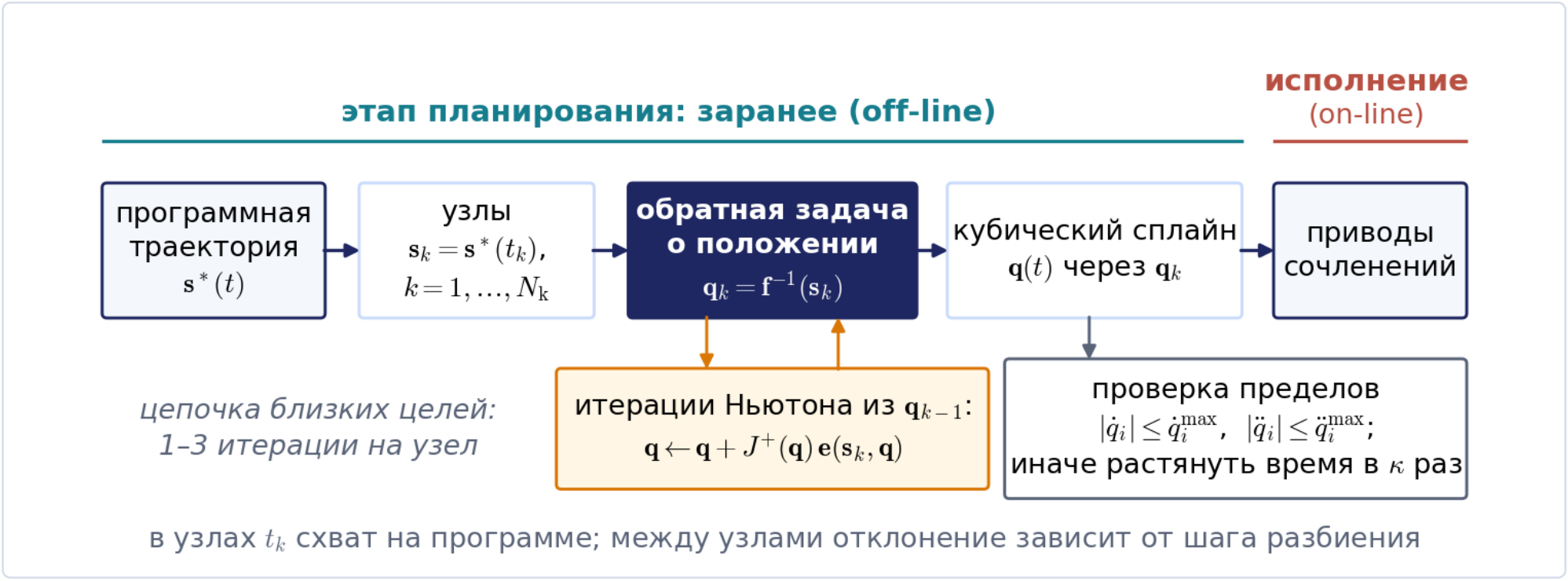
ЧАСТЬ 2

Управление в пространстве координат схвата

В первой части мы строили гладкие траектории в пространстве сочленений; теперь задание дано в координатах схвата, и его переводим в углы — по положению, скорости и силе.

5 Траектория схвата: обратная задача в узлах и сплайн между ними

Задание дано линией в координатах схвата; самый прямой путь — решить обратную задачу о положении в узлах этой линии.



- На траектории схвата выбирают N_k узлов, в каждом решают обратную задачу о положении и через решения q_k проводят сплайн в пространстве Q.
- Это **контурное управление со сплайн-интерполяцией**: тяжёлая работа сделана заранее, приводам остаётся отработать гладкую функцию.
- Сегодня численная обратная задача укладывается в такт: итерация Ньютона для UR5e в библиотеке kinlib занимает порядка миллисекунды.
- Метод выбирают по полноте и предсказуемости: аналитика (IKFast) даёт все ветви, итерации от прошлого узла сохраняют текущую ветвь.

5 Цепочка близких целей: обратная задача для UR5e

Старт итераций с решения для соседнего узла даёт быстрое схождение и не даёт решателю перепрыгнуть на другую ветвь.

2

итерации Ньютона на узел при старте от соседнего узла

≤ 3,3°

наибольший поворот сочленения между соседними узлами

8,9

итерации в среднем при старте каждого узла из нуля (до 22)

26 из 72

скачков больше 30° без цепочки; ещё в одном узле нет схождения

- Пример 4.6: UR5e из конфигурации (0; −90°; 90°; −90°; −90°; 0) обходит горизонтальную окружность радиусом 0,15 м с постоянной ориентацией.
- Для двухзвенника (пример 4.5) цепочка даёт 2,8 итерации на узел и сохраняет ветвь $q_2 > 0$, «локоть вниз»: q_2 меняется от 66° до 128°.
- Сплайн через узлы со скачками ветвей заставит робота размахивать звеньями — это главная типовая ошибка второго домашнего задания.

5 Прямая в пространстве сочленений — не прямая для схвата

Мало решить обратную задачу в двух крайних точках: равномерное движение углов уводит схват далеко от отрезка.



Двухзвенник с единичными звеньями идёт из A = (1,6; −0,5) в B = (0,3; 1,5). При линейном изменении углов схват уходит от отрезка между A и B на 0,54 м. Если поставить узлы на отрезке, ошибка падает как квадрат шага: 536 / 11,6 / 0,77 / 0,05 мм при 1 / 8 / 32 / 128 отрезках. Поэтому команда PTP (Point-to-Point, «из точки в точку») годится для переноса, а для шва нужна LIN (Linear, «по прямой»).

5 Ошибка хорды: сколько узлов ставить на окружности

Схват совпадает с программой только в узлах; между ними дуга заменена хордой, и густоту узлов задаёт допуск.

$$\varepsilon_{\text{ch}} = \rho\left(1 - \cos\frac{\Delta\varphi}{2}\right) \approx \frac{\rho\,\Delta\varphi^2}{8}, \qquad \Delta\varphi \le \sqrt{\frac{8\,\varepsilon_{\text{ch}}}{\rho}}$$

- Здесь ε_{ch} — наибольшее расстояние от дуги до хорды, ρ — радиус окружности, $\Delta\varphi$ — угловой шаг между соседними узлами.
- Ошибка растёт как квадрат шага: при ρ = 0,4 м и 36 узлах (шаг 10°) она равна 1,5 мм, а при 360 узлах — 0,015 мм.
- Пример 4.5: узлы, равномерные по времени, сгущаются у концов; в середине шаг 9,4°, и худшая хорда 1,3 мм против 0,38 мм при шаге 5°.
- Поэтому узлы ставят равномерно по дуге, а закон движения вдоль окружности задают отдельно, как в первой части.

6 Траектория схвата между двумя позами

Ориентацию нельзя интерполировать покомпонентно, поэтому перенос и поворот строят раздельно, но с одним законом движения.

1

$$T(t) = T_0^* A(t), \qquad A(t_0) = E, \qquad A(t_f) = (T_0^*)^{-1} T_f^*$$

Заданы начальная и конечная позы схвата T_0^* и T_f^*; ищем относительное перемещение A(t) из поворота R_A и переноса p_A.

2

$$\mathbf{p}(t) = \mathbf{p}_0 + \lambda(t)\,(\mathbf{p}_f - \mathbf{p}_0)$$

Перенос: точка схвата идёт по отрезку прямой с тем же законом λ(t), что и в первой части.

3

$$R_0^{\mathrm{T}} R_f = \mathrm{Rot}(\mathbf{k}, \alpha_f), \qquad \alpha(t) = \lambda(t)\,\alpha_f$$

Поворот: находим ось k и угол α_f полного поворота (занятие 1) и поворачиваем схват вокруг неподвижной оси на угол $\lambda(t)\alpha_f$.

4

$$R(t) = R_0 \exp\Big(\lambda(t)\ \log(R_0^{\mathrm{T}} R_f)\Big)$$

Та же кривая одной строкой — **геодезическая** в группе поворотов SO(3); в кватернионах её называют SLERP (Spherical Linear Interpolation).

При своих пределах для переноса и поворота длительность берут по большей из двух трапеций. В учебнике — (4.37)–(4.46); в (4.39), (4.41), (4.42) опечатки.

? Как думаете?

Схват нужно повернуть из ориентации R_0 в R_f. Студент предлагает $R(t) = (1 - \lambda)\cdot R_0 + \lambda\cdot R_f$: на концах ведь получаются R_0 и R_f. Что не так?

A Всё верно, нужно лишь взять λ пятой степени

B Промежуточная матрица — не поворот: оси схвата сжимаются и перестают быть ортогональными

C Неверно только направление: схват поворачивается длинным путём

? Как думаете?

Схват нужно повернуть из ориентации R_0 в R_f. Студент предлагает $R(t) = (1 - \lambda)\cdot R_0 + \lambda\cdot R_f$: на концах ведь получаются R_0 и R_f. Что не так?

A Всё верно, нужно лишь взять λ пятой степени

B Промежуточная матрица — не поворот: оси схвата сжимаются и перестают быть ортогональными

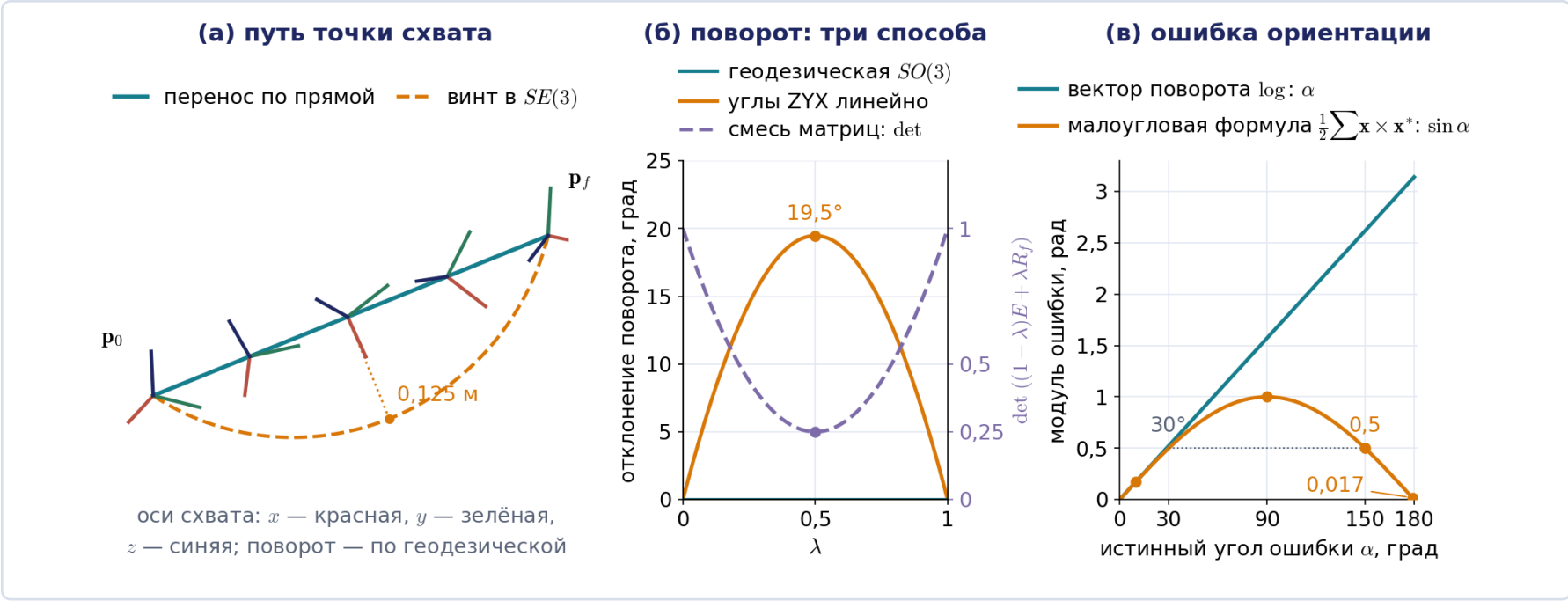
C Неверно только направление: схват поворачивается длинным путём

ОТВЕТ

Вариант B. При повороте на 120° в середине пути сингулярные числа матрицы (1; 0,5; 0,5), определитель 0,25, а при 180° ранг равен 1. Правильно: $R_0\cdot\exp(\lambda\cdot\log(R_0^{T}R_f))$.

6 Три способа повернуть схват (пример 4.7)

Сравним геодезическую с двумя способами, которые часто программируют по неопытности или по традиции.

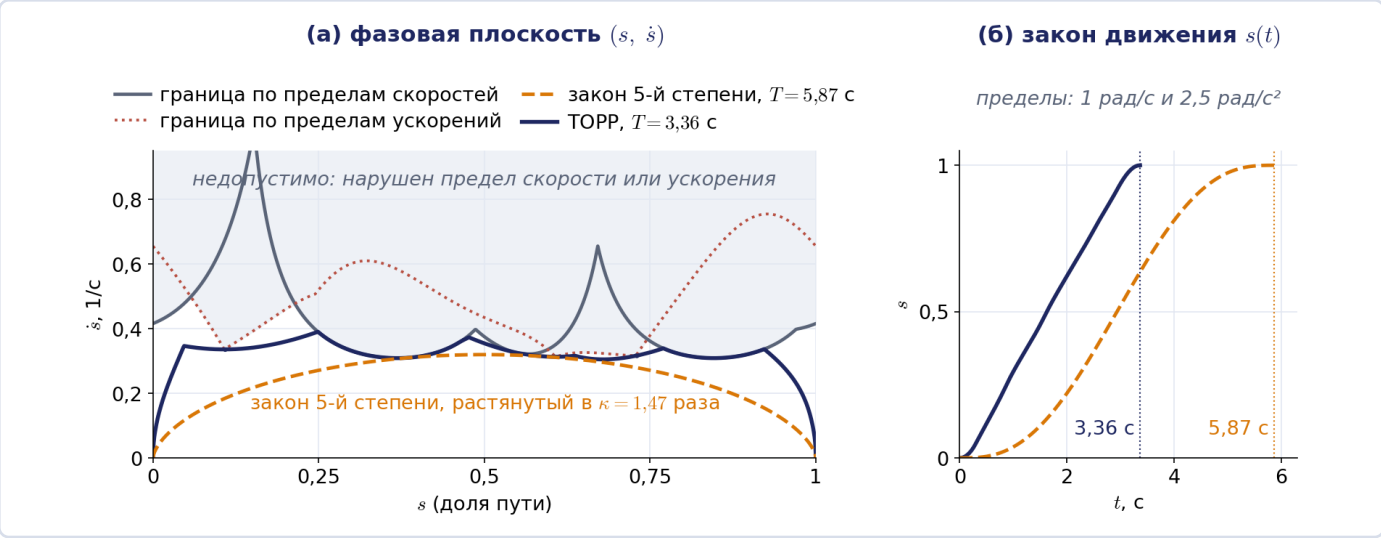


(а) путь точки схвата, (б) отклонение углов Эйлера и определитель смеси матриц, (в) модуль малоугловой ошибки против истинного угла.

- Пример 4.7: $R_0 = E$, конечная ориентация — поворот на 120° вокруг оси (1; 1; 1)/√3, точка схвата идёт из (0,5; 0; 0,3) в (0,3; 0,4; 0,5) м.
- Линейная интерполяция углов рыскания, тангажа и крена отходит от геодезической до 19,5° и поворачивает схват на 127° вместо 120°.
- Интерполяция позы по винту в группе движений SE(3) тоже геодезическая, но точка схвата уходит от отрезка длиной 0,49 м на 0,125 м.
- Малоугловая ошибка ½Σx×x* равна k·sin α: при 150° она та же, что при 30°, а у 180° почти нуль; для больших ошибок берут вектор поворота log.

6 Оптимальная по времени параметризация пути (TOPP)

Растяжение замедляет весь путь из-за одного тяжёлого участка; быстрее выбрать закон движения вдоль пути под пределы.



TOPP — Time-Optimal Path Parameterization; TOPP-RA — её алгоритм через анализ достижимости; TOTG — Time-Optimal Trajectory Generation.

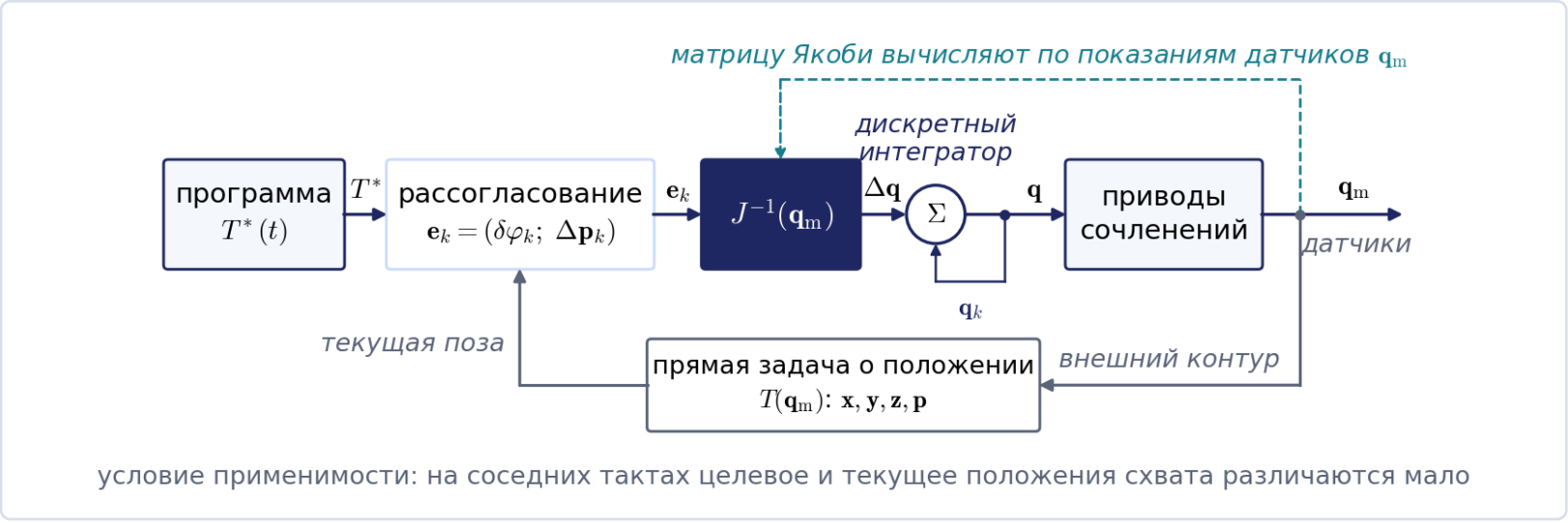
- Путь q(s) задан, ищем закон s(t): пределы скоростей и ускорений дают на фазовой плоскости (s, ṡ) кривую предельной скорости.
- Оптимум устроен по принципу «газ — тормоз»: наибольший разгон, движение вдоль кривой предела, наибольшее торможение; время равно ∫ds/ṡ.
- TOPP-RA (Фам и Фам, 2018) делает два прохода с малой задачей линейного программирования на шаге; в MoveIt 2 по умолчанию — родственный TOTG.
- Пример 4.8: окружность примера 4.5 при пределах 1 рад/с и 2,5 рад/с² — 3,36 с вместо 5,87 с при растяжении, на 43 % быстрее.

$$\dot{\mathbf{q}} = \mathbf{q}'(s)\,\dot{s}$$

$$\ddot{\mathbf{q}} = \mathbf{q}'(s)\,\ddot{s} + \mathbf{q}''(s)\,\dot{s}^2$$

7 Линеаризованный позиционный алгоритм: одна итерация Ньютона на такт

Если цель за такт сдвигается мало, обратную задачу можно не решать: достаточно одной итерации Ньютона в каждом такте.



В учебнике — формулы (4.47)–(4.53); там рассогласование обозначено Δs_k.

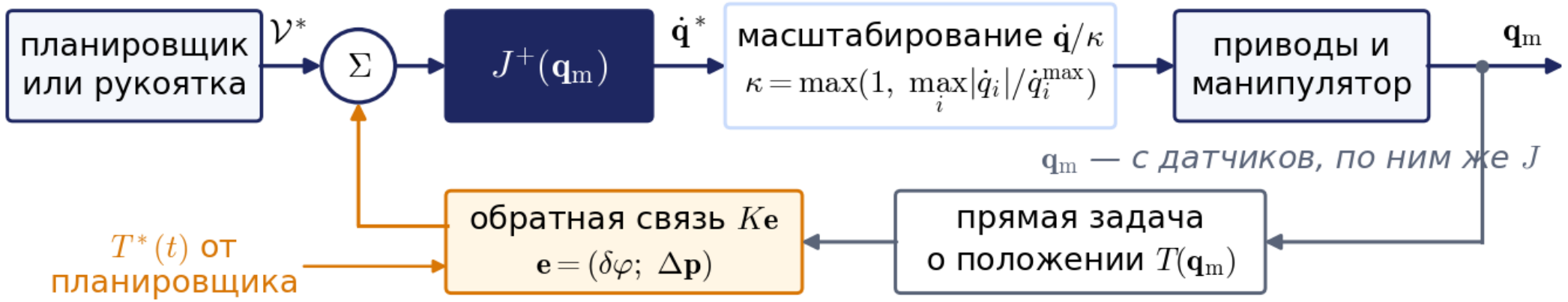
- Вектор e_k из ошибки ориентации $\delta\varphi_k$ и ошибки положения Δp_k — это малое перемещение, которое снимает рассогласование за один такт Δt.
- Решатель обратной задачи становится следящей системой: это цепочка близких целей, перенесённая с этапа планирования в реальное время.
- Положение схвата считают по датчикам q_m; при приводах, замкнутых по скорости, $\Delta q/\Delta t$ подают как скорость, по положению — суммируют.
- Обратная задача не решается, зато в каждом такте обращается матрица J, поэтому алгоритм чувствителен к вырожденным конфигурациям.

$$\Delta\mathbf{q}_{k+1} = J^{-1}(\mathbf{q}_k)\,\mathbf{e}_k$$

$$\mathbf{e}_k = \begin{pmatrix} \delta\boldsymbol{\varphi}_k \\ \Delta\mathbf{p}_k \end{pmatrix}$$

8 Управление по скорости с обратной связью по положению (CLIK)

Без обратной связи любая ошибка интегрирования или привода накапливается; добавим к командной скорости поправку по ошибке позы.



Без обратной связи, $\dot{q} = J^{-1}\mathcal{V}^*$, схему называют RMRC (Resolved Motion Rate Control, Уитни, 1969).

- **CLIK** (Closed-Loop Inverse Kinematics, обратная кинематика с замкнутым контуром): $\mathcal{V}^*$ — прямая связь по скорости, Ke — по положению.
- При полном ранге $J \cdot J^+ = E$, и ошибка положения затухает с постоянной времени $1/K$ в любой конфигурации; для ориентации — при малых ошибках.
- Функция Ляпунова $V = \tfrac{1}{2} e^T e$ убывает: её производная равна $-e^T K e < 0$ при ненулевой ошибке, и это рассуждение годится для нелинейных систем.
- Без прямой связи (это и есть линеаризованный алгоритм, $K = 1/\Delta t$) схват отстаёт на $\|\mathcal{V}^*\|/K$: на окружности при $K = 20\ с^{-1}$ — на 59 мм.

$$\dot{\mathbf{q}} = J^+(\mathbf{q})\left(\boldsymbol{\nu}^* + K\mathbf{e}\right)$$

$$\dot{\mathbf{e}} = -K\mathbf{e}, \qquad \mathbf{e}(t) = e^{-Kt}\,\mathbf{e}(0)$$

8 Алгоритм с транспонированной матрицей Якоби

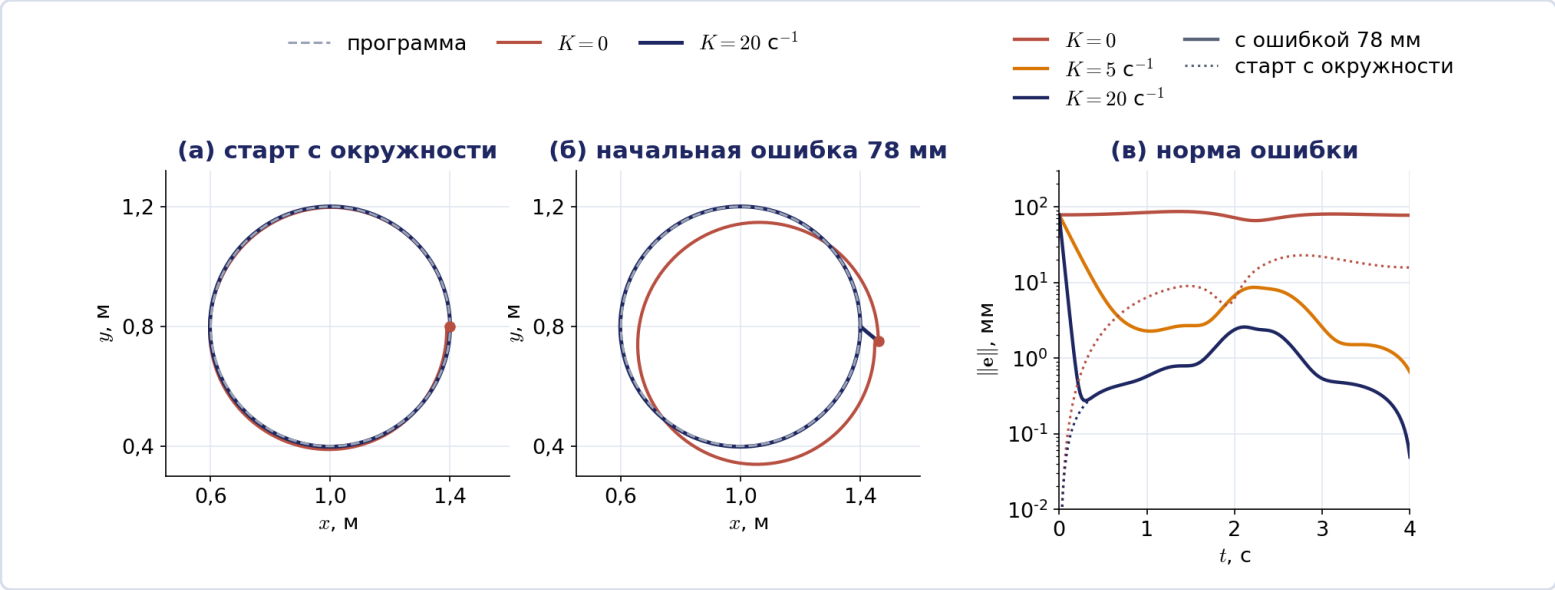
Функция Ляпунова подсказывает закон, в котором матрицу Якоби вообще не нужно обращать.

- Для неподвижной цели скорости сочленений берут пропорциональными $J^T e$ с числом $K_T > 0$; производная V не положительна, ошибка не растёт.
- Плата — медленная и неравномерная сходимость: вдоль i-го сингулярного направления ошибка убывает со скоростью $K_T \cdot \sigma_i^2$.
- Пример 4.5: σ = 1,82 и 0,52, при K_T = 20 скорости убывания 66,5 и 5,5 $с^{-1}$; из смещения 78 мм до 1 мм — 0,53 с против 0,22 с у закона с J^+.
- Если ошибка лежит в ядре J^T, алгоритм застревает; $J^T(Ke)$ — моменты от пружины между схватом и целью, отсюда мост к управлению по импедансу.

$$\dot{\mathbf{q}} = K_T\, J^{\mathrm{T}}\mathbf{e}, \qquad \dot{V} = -K_T\, \mathbf{e}^{\mathrm{T}} J J^{\mathrm{T}}\mathbf{e} \le 0$$

8 Цифровая реализация и выбор коэффициента (пример 4.9)

В цифровой системе коэффициент нельзя увеличивать беспредельно: такт и запаздывание ограничивают его сверху.



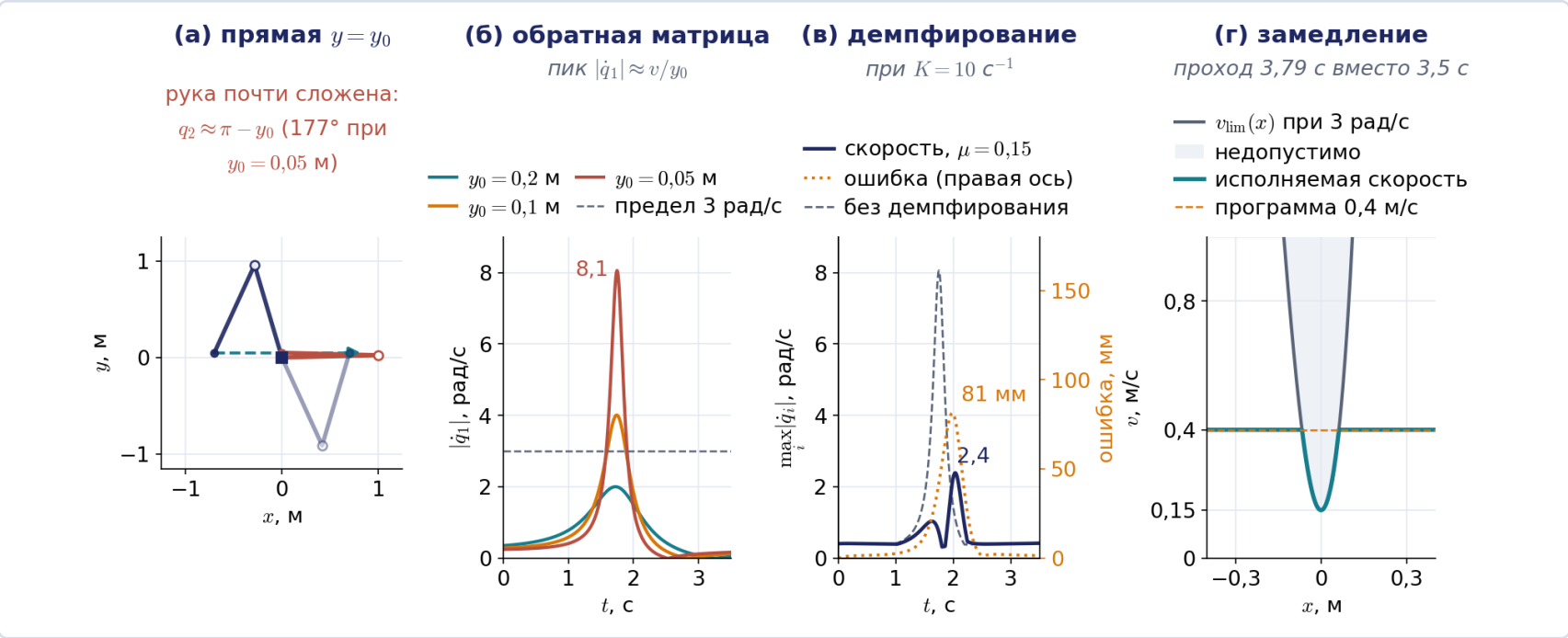
- Такт 0,02 с: без обратной связи ошибка доходит до 23 мм, при K = 5 и 20 с$^{-1}$ — до 8,6 и 2,6 мм; смещение 78 мм за 0,2 с падает до 0,6 мм.
- Ошибка за такт умножается на 1 − KΔt: при KΔt < 1 она убывает монотонно, при 1 < KΔt < 2 — с колебаниями, при KΔt > 2 контур неустойчив.
- Запаздывание на такт сужает устойчивость до KΔt < 1, а без колебаний — до KΔt ≤ 1/4; при частоте 500 Гц это K < 500 и K ≤ 125 с$^{-1}$.
- Реальный предел ещё ниже: его задают полоса внутреннего контура привода и шум датчиков; у команды servoj в URScript аналог K — усиление.

$$\mathbf{e}_{k+1} = (1 - K\Delta t)\,\mathbf{e}_k$$

$$\mathbf{e}_{k+1} = \mathbf{e}_k - K\Delta t\,\mathbf{e}_{k-1}: \quad z^2 - z + K\Delta t = 0$$

9 Проход рядом с особой точкой

Все алгоритмы с обращением J наследуют беду обратной задачи о скорости: у особой точки скорости сочленений взлетают.



– Схват идёт со скоростью v = 0,4 м/с по прямой y = y_0 мимо основания; в ближайшей точке рука почти сложена: $q_2 \approx \pi - y_0$ (177° при 0,05 м).
– Там $\dot{q}_2 = 0$, а первое сочленение следит за направлением на схват: $|\dot{q}_1| = v/y_0$, то есть 2,0 / 4,0 / 8,1 рад/с при y_0 = 0,2 / 0,1 / 0,05 м.
– Причина видна из сингулярного разложения: наименьшее сингулярное число J около y_0, а команда направлена вдоль этого «слабого» направления.
– Демпфирование с μ = 0,15 при K = 10 $с^{-1}$ снижает пик до 2,4 рад/с, но схват на время отстаёт от программы на 81 мм.

9 Демпфирование и масштабирование: чем платим

Скорости у особой точки ограничивают разными способами, и каждый платит своим: точностью, отставанием или временем.

Демпфированная псевдообратная

- Усиление $\sigma/(\sigma^2 + \mu^2)$ не больше $1/(2\mu)$, поэтому скорости ограничены в любой конфигурации.
- Плата — точность: у особой точки схват отрабатывает команду не полностью и отходит от программы.
- Коэффициент μ делают переменным: нулевым вдали от особых точек и растущим при приближении к ним.
- MoveIt Servo по числу обусловленности J сначала замедляет движение, затем останавливает (пороги 17 и 30).

$$\dot{\mathbf{q}} = J^{\mathrm{T}}(JJ^{\mathrm{T}} + \mu^2 E)^{-1}(\boldsymbol{\mathcal{V}}^* + K\mathbf{e})$$

Масштабирование скоростей и времени

- Срезать скорости по отдельности нельзя: меняется направление, и схват сходит с прямой на 38 мм.
- Общий делитель κ сохраняет направление, но программа уходит вперёд: отставание достигает 115 мм.
- Замедление программы (масштабирование времени): уход и отставание до 0,3 мм, проход 3,79 с вместо 3,5 с.
- Это движение по кривой предельной скорости, частный случай оптимальной параметризации; на пульте — override.

$$\dot{\mathbf{q}} \leftarrow \frac{\dot{\mathbf{q}}}{\kappa}, \qquad \kappa = \max\left(1,\ \max_i \frac{|\dot{q}_i|}{\dot{q}_i^{\max}}\right)$$

9 Ограничения как задача квадратичного программирования (QP)

Демпфирование, общий множитель и вторичные задачи — частные случаи одной постановки: малой задачи оптимизации в каждом такте.

- Без ограничений минимум первой задачи даёт ровно демпфированную псевдообратную матрицу с тем же коэффициентом μ.
- Во второй задаче схват движется строго в заданном направлении, но не быстрее команды; при одних пределах скоростей β = 1/к.
- Так устроены функция DoDifferentialInverseKinematics пакета Drake (при очень малом β она сообщает о застревании) и библиотеки Pink и mink.
- Такой слой удобен и тогда, когда позу или скорость схвата выдаёт обученная нейросеть, а углы сочленений вычисляет QP.

$$\min_{\dot{\mathbf{q}}} \|J\dot{\mathbf{q}} - \boldsymbol{\mathcal{V}}^*\|^2 + \mu^2\|\dot{\mathbf{q}}\|^2$$

$$\max \beta: \quad J\dot{\mathbf{q}} = \beta\,\boldsymbol{\mathcal{V}}^*, \quad 0 \le \beta \le 1, \quad |\dot{q}_i| \le \dot{q}_i^{\max}, \quad \mathbf{q}^{\min} \le \mathbf{q} + \dot{\mathbf{q}}\,\Delta t \le \mathbf{q}^{\max}$$

9 Застревание в особой точке

Может ли манипулятор, оказавшийся в вырожденной конфигурации, не выйти из неё? Может, и это легко показать.

- Двухзвенник с единичными звеньями вытянут вдоль оси x, q = (0; 0); команда велит двигать схват вдоль руки к основанию со скоростью 0,2 м/с.
- Столбцы J перпендикулярны руке, ранг равен 1, а $J^{+} = J^{T}/\sigma_1^2$; команда лежит в ядре J^{T}, поэтому $\dot{q} = 0$ и рука стоит, хотя цель достижима.
- Демпфирование и алгоритм с транспонированной матрицей не помогают: в них тот же множитель J^{T}.
- Выход — движение в пространстве сочленений: уже при сгибе локтя на 0,2 рад та же команда даёт $\dot{q} = (-0{,}99;\ 1{,}99)$ рад/с.
- То же ядро J^{T} разгружает приводы, когда сила направлена вдоль вытянутой руки: где схват «быстр», там он «слаб», и наоборот.

$$J(\mathbf{0}) = \begin{pmatrix} 0 & 0 \\ 2 & 1 \end{pmatrix}, \qquad \boldsymbol{\mathcal{V}}^{*} = \begin{pmatrix} -0{,}2 \\ 0 \end{pmatrix}\ \text{м/с}, \qquad J^{\mathrm{T}}\boldsymbol{\mathcal{V}}^{*} = \mathbf{0}$$

Кейс: шов через кистевую особенность

Команда движения по прямой (LIN, MoveL) через кистевую особую конфигурацию останавливается с ошибкой, а команда «из точки в точку» (PTP, MoveJ) в ту же точку выполняется. Почему?

A У команды «из точки в точку» выше допустимые скорости приводов

B По прямой обратная задача решается в каждой точке пути, и у особой точки запястью нужны неограниченные скорости

C Команда «из точки в точку» использует демпфированную псевдообратную матрицу

D Конечная точка недостижима для движения по прямой

Кейс: шов через кистевую особенность

Команда движения по прямой (LIN, MoveL) через кистевую особую конфигурацию останавливается с ошибкой, а команда «из точки в точку» (PTP, MoveJ) в ту же точку выполняется. Почему?

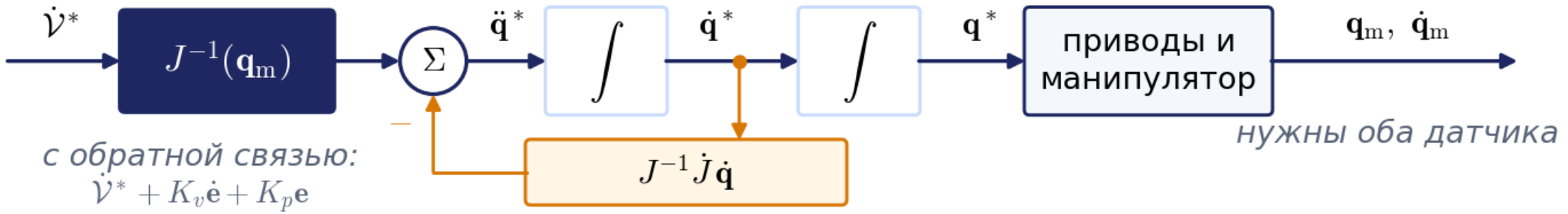
- **A** У команды «из точки в точку» выше допустимые скорости приводов
- **B** По прямой обратная задача решается в каждой точке пути, и у особой точки запястью нужны неограниченные скорости
- **C** Команда «из точки в точку» использует демпфированную псевдообратную матрицу
- **D** Конечная точка недостижима для движения по прямой

ОТВЕТ

Вариант B. «Из точки в точку» решает обратную задачу только в конечной точке, а путь схвата не контролирует. Для движения по прямой у производителей есть режимы с допуском по ориентации.

10 Управление по вектору ускорения

Продифференцируем связь скоростей ещё раз и получим закон, который дальше станет входом уравнений динамики.



В учебнике — формула (4.57); ускорение схвата там обозначено $\ddot{s}$ = (ε; a).

- Командой служит ускорение схвата как твёрдого тела, то есть производная $\mathcal{V}^*$; закон даёт ускорения сочленений $\ddot{q}$, их дважды интегрируют.
- Слагаемое с производной J — центростремительные и кориолисовы составляющие: схват ускоряется даже при $\ddot{q} = 0$, потому что меняется J.
- Нужны датчики не только положений q_m, но и скоростей $\dot{q}_m$, а производную J приходится вычислять в каждом такте.
- Ускорения сочленений — естественный вход динамики; с обратной связью к команде добавляют $K_v \cdot \dot{e} + K_p \cdot e$ (Лух, Уокер и Пол, 1980).

$$\dot{\mathcal{V}} = J\ddot{\mathbf{q}} + \dot{J}\dot{\mathbf{q}}$$

$$\ddot{\mathbf{q}} = J^{-1}\left(\dot{\mathcal{V}}^* - \dot{J}\dot{\mathbf{q}}\right)$$

10 Управление по вектору силы: $\tau = J^T F$

Последний мост связывает силы: цель здесь не движение, а взаимодействие схвата со средой.

1

$$\mathbf{F} = (\mathbf{m};\, \mathbf{f})$$

F — силовой винт: момент и сила, которые схват прикладывает к среде; среда давит на схват силой −F, моменты τ в сочленениях её уравновешивают.

2

$$\boldsymbol{\tau}^{\mathrm{T}} \delta\mathbf{q} + (-\mathbf{F})^{\mathrm{T}} \delta\mathbf{s} = 0, \qquad \delta\mathbf{s} = J\,\delta\mathbf{q}$$

Принцип виртуальных перемещений: в равновесии работа приводов на любом малом перемещении δq равна работе схвата над средой на δs.

3

$$\boldsymbol{\tau} = J^{\mathrm{T}}(\mathbf{q})\,\mathbf{F}$$

Так как δq произвольно, моменты равны $J^T F$; обращать ничего не нужно, и формула работает даже в вырожденной конфигурации.

4

$$\hat{\mathbf{F}} = (J^{\mathrm{T}})^{+}\boldsymbol{\tau}_{\mathrm{ext}}$$

Формула в обратную сторону: по внешним моментам в сочленениях кобот оценивает силу на схвате; у особой точки оценка ненадёжна из-за ядра J^T.

Пример учебника: двухзвенник держит крышку люка силой P/2, τ = (−Pl/2; −Pl/4); в учебнике для τ_1 напечатан знак плюс, моменты обозначены μ_i.

11 Сводка методов кинематического управления

Соберём все способы в одну таблицу: три выполняются заранее, четыре — в каждом такте управления.

Метод	Команда и ключевая формула	Когда считается	Особенности
Из точки в точку	q^o, q^1; $q = q^o + \lambda(t)(q^1 - q^o)$	заранее	путь схвата не контролируется; сочленения синхронны
Сплайн и параметризация по времени	q_k, t_k; трёхдиагональная система; фазовая плоскость $(s, \dot{s})$	заранее	непрерывное ускорение; наименьшее время при пределах
Позиционное (контурное)	$T^*(t)$; $q_k = f^{-1}(s_k)$ и сплайн	заранее	особые точки не мешают исполнению; ошибка хорды
Линеаризованное позиционное	$T^*(t)$; $\Delta q = J^{-1}e$	в каждом такте	без обратной задачи; чувствительно к особым точкам
По скорости с обратной связью	$\mathcal{V}^*$; $\dot{q} = J^+(\mathcal{V}^* + Ke)$	в каждом такте	ошибка затухает как e^{-Kt}; устойчиво при $K\Delta t < 2$
По вектору ускорения	ускорение схвата; $\ddot{q} = J^{-1}\cdot(\text{команда} - (dJ/dt)\cdot\dot{q})$	в каждом такте	нужны датчики скорости и dJ/dt; мост к динамике
По вектору силы	F^*; $\tau = J^T F$	в каждом такте	транспонирование вместо обращения; статика

Все методы считают приводы идеальными; для быстрых и точных движений нужна динамика, и управление по ускорению и по силе — готовые мосты к ней.

11 Где это в реальной системе: от формул к контроллеру робота

За каждой командой промышленного робота и ROS 2 стоит формула этой лекции; названия различаются, смысл один.

Метод лекции	Команды KUKA, ABB, FANUC, UR	ROS 2 и MoveIt 2
Из точки в точку, синхронизация (4.2)	PTP, MoveJ, J, movej; флаги ветви Status/Turn, confdata	PTP планировщика Pilz: трапеция, ведущее сочленение
Прямая и поворот по геодезической (4.6)	LIN, MoveL, L, movel	LIN: перенос по прямой, поворот по SLERP
Дуга окружности (второе домашнее задание)	CIRC, MoveC, C, movec	CIRC по центру или промежуточной точке
Скругление угла ломаной (4.4)	зоны скругления: ABB z10 против точного останова fine	радиус скругления blend_radius в последовательности Pilz
Масштабирование времени (4.9)	коррекция скорости (override) на пульте	масштабы скорости и ускорения в запросе на планирование
Многочлены, параметризация, рывок (4.3, 4.6)	—	joint_trajectory_controller: степени 1, 3, 5; TOTG и Ruckig
Управление по скорости (4.8)	перемещение кнопками пульта; speedl и servoj у UR	MoveIt Servo

ROS 2 — Robot Operating System 2; PTP, LIN, CIRC — Point-to-Point, Linear, Circular; SLERP — сферическая линейная интерполяция; TOTG — Time-Optimal Trajectory Generation. Синтаксис — в методических указаниях к лабораторным.

11

Кейс «горелка по фланцу»: контрольный список

Вернёмся к кейсу из начала занятия: всё, что нужно помнить из лекции, собрано в пять пунктов проверки.

- Плавный старт и останов: закон движения вдоль окружности задают профилем разгона — торможения, лучше с ограниченным рывком, а не ступенчатыми уставками.
- Форма пути: узлы ставят на окружности равномерно по дуге, а их число выбирают по допуску ошибки хорды ε_{ch}.
- Ветвь и гладкость: обратную задачу решают цепочкой от соседнего узла, через решения проводят сплайн (для повторного обхода — периодический).
- Время и точность: время обхода даёт параметризация по времени, а на исполнении работает обратная связь с прямой связью по скорости.
- Особые точки: окружность ставят вдали от них, иначе применяют демпфирование и масштабирование времени и следят, чтобы команда не легла в ядро J^T.

Выводы

1. Кинематическое управление строит программное движение $q^*(t)$ или $T^*(t)$ при идеальных приводах; сегодня это параметризация по времени и контроллер траектории.

2. Путь и закон движения задают раздельно; трапецию считают по пределам привода и синхронизируют сочленения по ведущему.

3. Гладкость нужна против конкретных собственных частот: S-кривая платит лишь временем t_j, многочлен пятой степени — почти полуторным временем.

4. Через много точек проводят кубический сплайн; при нарушении пределов время растягивают в κ раз или параметризуют путь оптимально на фазовой плоскости.

5. Форму пути задают узлами на траектории схвата с цепочкой обратных задач и допуском хорды, а ориентацию ведут по геодезической.

6. Закон $\dot{q} = J^{+}(\mathcal{V}^* + Ke)$ даёт экспоненциальное убывание ошибки; в цифровом контуре он устойчив при $K\Delta t < 2$ и не колеблется при $K\Delta t < 1$.

7. У особых точек скорости растут как v/σ_{min}: демпфирование платит точностью, общий делитель — отставанием, масштабирование времени — временем.

Вопросы для самопроверки (в скобках — номер в конспекте)

1. Почему движение с постоянной скоростью неприемлемо и какому условию подчиняется профиль скорости $d\lambda/dt$? (2)

2. Рассчитайте трапецию для $\dot{q}^{max} = 2$ рад/с, $\ddot{q}^{max} = 8$ рад/с² и перемещений 3 и 0,2 рад. (3)

3. Что такое S-кривая и почему ограничение рывка гасит одни собственные частоты и не влияет на другие? (5)

4. Почему через много точек проводят сплайн, а не многочлен, и как исправить нарушение пределов? (7, 9)

5. Зачем решать обратную задачу цепочкой близких целей? Сколько узлов нужно на окружности $\rho = 0{,}3$ м при $\varepsilon_{ch} = 0{,}1$ мм? (10)

6. Почему нельзя интерполировать элементы матрицы поворота или углы Эйлера и чем раздельная интерполяция отличается от винтовой? (11)

7. Покажите функцией Ляпунова, что закон $\dot{q} = J^{+}(\mathcal{V}^{*} + Ke)$ уменьшает ошибку. Чем отличается алгоритм с J^{T}? (13)

8. Может ли манипулятор не выйти из вырожденной конфигурации? Как ограничивают скорости у особой точки? (14)

9. Кейс: контур 500 Гц, интегрирование по Эйлеру. Каков предел K без запаздывания и с запаздыванием на такт? (16)

10. Кейс: программа подаёт 72 точки окружности $\rho = 0{,}2$ м ступенями каждые 0,08 с. Какова ошибка хорды и что изменить? (17)